

How Do Different Neural Architectures Reason?

A Case Study of Sudoku Solving

CHEN, Ruiping

Hong Kong University of Science and Technology
Clear Water Bay, Kowloon, Hong Kong

rchenbt@connect.ust.hk

Sharing consent: I do **not** consent to sharing this report with other students in the course (no impact on grading).

Abstract

Sudoku asks for a placement of digits on a $k \times k$ grid so that every row, column, and each $\sqrt{k} \times \sqrt{k}$ block contains $\{1, \dots, k\}$ exactly once (classic puzzles use $k=9$). Classical exact solvers—including dancing-links formulations (DLX) and explicit constraint satisfaction (CSP) search—can reach perfect accuracy but, for generalized puzzles, face worst-case complexity that grows sharply with instance size, making exhaustive methods increasingly impractical for large variants. In contrast, a fixed-depth neural predictor’s inference time scales polynomially in the number of cells for standard layers (e.g., convolutions, message passing, or self-attention with polynomial complexity in sequence length), offering a complementary “amortized reasoning” viewpoint focused on accuracy and scalability rather than certified optimality.

We therefore treat Sudoku as a compact testbed for comparing how different neural architectures propagate constraints and interact with supervision versus sequential decision making. We study several supervised backbones with shared training protocol and a PPO-based reinforcement-learning agent with constraint-aware masking. On $N=20,000$ puzzles and 50 supervised epochs, a GCN reaches 97.2% blank-cell validation accuracy on `sudoku-1m`, closely followed by a Transformer at 95.6%; MLP and CNN lag (55.8% and 30.6%). Training on difficulty-stratified `sudoku-3m-easy` and `sudoku-3m-hard` exports reveals a large distribution shift: blank accuracy falls to roughly 53%–58% for the best architectures and below 33% for MLP/CNN. Guided PPO (MRV + candidate mask) reaches 99.97% blank-cell accuracy on `sudoku-1m`, but removing either heuristic in a 2×2 ablation collapses accuracy to $\approx 14\%$ – 19% ; training on the full `sudoku-3m` subsample still drops under shift (26.7%), paralleling supervised models. We interpret these results through inductive bias, clue-count shift, and

constraint-aware sequential policies.

1. Introduction

Problem and classical solvers. The objective of Sudoku is to fill a $k \times k$ grid with digits so that each column, each row, and each of the k subgrids (with size $\sqrt{k} \times \sqrt{k}$) contains all digits from 1 to k exactly once. In classic Sudoku, $k=9$. Highly engineered *exact* algorithms—notably Knuth’s *dancing links* (DLX) formulation and general CSP/backtracking search with constraint propagation—can achieve 100% accuracy on valid instances by systematically exploring the discrete search space [2].

Why neural solvers? For generalized puzzle families, Sudoku completion is NP-complete [11], and deterministic search can exhibit sharply growing worst-case cost as board size increases. We study *neural* solvers as predictors whose fixed-depth **forward pass** runs in time polynomial in the number of cells: they trade certified completeness for amortized learning, paying training cost once and then deploying cheap per-puzzle inference.

Why Sudoku as a testbed. Because rules are crisp and inputs are structured grids, Sudoku isolates *how different neural architectures propagate constraints* without raw-sensory confounders. We compare approaches under a shared recipe on public 9×9 CSV corpora (`sudoku-1m` [7] and difficulty-stratified `sudoku-3m` subsets; Sec. 3).

Approach. We train four supervised one-shot predictors—MLP, CNN, Transformer, and GCN—plus a PPO policy with constraint-aware action masking (Sec. 4). All models share preprocessing and optimization defaults where possible (Sec. 5).

Key results. On $N=20,000$ puzzles and 50 supervised epochs, the GCN reaches 97.2% blank-cell validation accuracy on `sudoku-1m`, closely followed by a Transformer at 95.6%; MLP and CNN lag (55.8% and 30.6%). On `sudoku-3m-easy` and `sudoku-3m-hard`, the best architectures fall to roughly 53%–58%, exposing distribution shift. Guided PPO (MRV + candidate mask) attains 99.97% blank accuracy on `sudoku-1m`, but a 2×2 ablation shows that removing either heuristic drops accuracy to $\approx 14\%$ – 19% . PPO trained on a full `sudoku-3m` subsample degrades to 26.7% blank accuracy, paralleling supervised models under shift.

Contributions. We (i) frame classical search versus polynomial-time neural inference; (ii) compare four supervised inductive biases and PPO under aligned data and metrics; (iii) ablate supervised loss scope and PPO guidance; and (iv) contextualize against published diffusion+RL Sudoku numbers where metrics align (Sec. 5.3).

2. Related Work

Neural approaches to Sudoku and similar CSPs. Sudoku has repeatedly been used to evaluate relational reasoning models. Recurrent Relational Networks [6] message-pass over “digits that must differ” relationships and achieve strong puzzle-level accuracy, closely related to our GCN view. More broadly, graph networks emphasize encoding constraints as edges and learned message functions [1], which motivates treating each cell as a node connected to its Sudoku peers.

Convolutional and attention models. Convolutional networks impose locality and translation structure [3], which can help when clues propagate through neighborhoods but may under-express long-range constraints without depth or dilation. Transformers remove locality assumptions and can propagate information between arbitrary cells in one layer [10], which is attractive when constraints are global.

Reinforcement learning for combinatorial tasks. Policy-gradient methods such as PPO [9] stabilize learning in discrete action spaces and have been applied broadly to games and control [5]. For Sudoku, RL is appealing because invalid placements can be penalized immediately, but sample efficiency and credit assignment across long horizons remain challenging compared to supervised imitation.

Diffusion models as neuro-symbolic reasoners. Diffusion models are increasingly used beyond continuous generation by treating iterative denoising as sequential decision

making. Zhang et al. [12] propose **DDReasoner**, a two-stage pipeline for logical puzzles: stage 1 trains a masked DDPM for basic solution generation; stage 2 formulates denoising as an MDP and fine-tunes with policy optimization using a *rule-based* outcome reward that checks hard logical consistency of the final discretized board. On Sudoku, they report large gains from this constraints-guided RL stage over supervised diffusion alone, measured by the fraction of *fully correct* boards on several public datasets. Their work is publicly described [12] and provides a natural external reference point for our simpler PPO agent that acts directly on cell placements.

Classical constraint satisfaction and exact search. Sudoku is a finite-domain CSP and is solved exactly by backtracking, arc consistency, SAT reductions, and highly engineered algorithms such as DLX [2]. For generalized boards, worst-case search can blow up in instance size [11]. Neural networks do not replace these guarantees in our setup; instead they provide a different operating point: *amortized* predictions with polynomial-time forward passes for fixed architectures.

Positioning. Our goal is not to beat dedicated classical solvers on 9×9 instances, nor to claim neural networks are “better” in every sense. Rather, we use Sudoku to study *how* different neural architectures approximate constraint propagation, and how that interacts with scalability across puzzle sizes and learning paradigms (supervision vs. RL).

3. Data

Scope. All quantitative experiments in this report use **standard 9×9 Sudoku** from public CSV corpora: (i) a million-puzzle Kaggle release [7] denoted `sudoku-1m` in this report; (ii) the larger three-million-puzzle Kaggle release [8] denoted `sudoku-3m`, with metadata columns `clues` and `difficulty`; and (iii) two difficulty-stratified subsets we extracted from `sudoku-3m`, `sudoku-3m-easy` and `sudoku-3m-hard` (Sec. 3). We do not evaluate other board sizes in this report.

Easy/hard split from `sudoku-3m`. We filter the full `sudoku-3m` table with two *independent* rules (a puzzle may appear in neither, one, or both exports). **Easy** rows must satisfy *both* conditions: `difficulty` ≤ 1.5 and `clues` ≥ 25 . **Hard** rows satisfy *either* condition: `difficulty` ≥ 2.5 or `clues` ≤ 22 . We apply these thresholds to the dataset’s own `difficulty` scalar together with the number of given cells as a structural proxy. Under this recipe we obtain 727,892 easy and 662,385 hard puzzles (about 24% and 22% of the 3×10^6 parent file, with

Puzzle (quizzes)									Solution								
		4	3			2		9	8	6	4	3	7	1	2	5	9
		5				9			3	2	5	8	4	9	7	6	1
	7				6				9	7	1	2	6	5	8	4	3
		6				2			4	3	6	1	9	2	5	8	7
1	9					7	4		1	9	8	6	5	7	4	3	2
		5				8	3		2	5	7	4	8	3	9	1	6
6								1	6	8	9	7	3	4	1	2	5
			3	5		8	6	9	7	1	3	5	2	8	6	9	4
	4	2	9	1		3			5	4	2	9	1	6	3	7	8

Figure 1. Example sudoku-1m row. Left: quizzes string with blanks as empty cells; right: solutions string. Bold digits are given in the puzzle; regular weight in the solution panel marks filled blanks.

overlap between the two sets possible). The easy and hard exports are not mutually exclusive by construction.

Format and preprocessing. Each row provides an 81-character puzzle string and a matching 81-character solution string in row-major order. In sudoku-1m, columns are quizzes and solutions; blanks in the puzzle are encoded as 0. In sudoku-3m, columns are puzzle and solution; blanks use the dot character ., which we normalize to 0 before tensorization so that all models share the same input pipeline. Digits are characters 1–9. We map strings to 9×9 integer tensors. For supervised models, clues use a 10-way one-hot per cell (blank plus nine digits), yielding a tensor of shape $(10, 9, 9)$, equivalently $(k+1, k, k)$ with $k=9$ in the notation of Sec. 4. Targets use nine classes per cell aligned with digits 1–9 (blank clue cells are still supervised using the solution digit). Fig. 1 shows a typical sudoku-1m row: the left board is the input puzzle with 0 for blanks (bold digits are given clues); the right board is the matching complete solution.

Splits and scale. Unless stated otherwise, we subsample $N=20,000$ puzzles per corpus for training experiments (Sec. 5) and reserve 20% of that subsample for validation. The full sudoku-1m CSV contains 10^6 rows; sudoku-3m contains 3×10^6 (see Sec. 3 for easy/hard export sizes). We do not train on the full files within the course budget, but the protocol scales to larger subsamples.

Clue-count distributions. The number of given cells (*clues*) is a coarse proxy for puzzle hardness: fewer clues usually imply more search. Fig. 2 summarizes the three corpora used in our experiments (full files). The sudoku-1m set is tightly concentrated between 29 and 37 clues (mean 33.8, std 0.9), with a sharp mode at 34 clues (45.5% of puzzles). This differs markedly from sudoku-3m-easy and sudoku-3m-hard: after the split in Sec. 3, the easy subset clusters at 25–26 clues (70.1% and 25.2%), while the

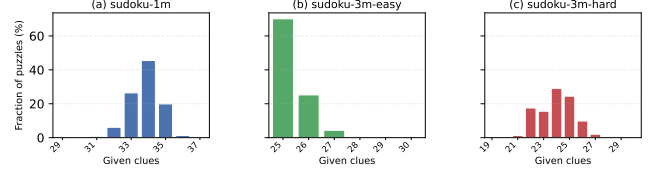


Figure 2. Clue-count distributions for sudoku-1m, sudoku-3m-easy, and sudoku-3m-hard (percentage of puzzles per bin; full files). Easy/hard subsets are defined in Sec. 3.

hard subset spans 22–25 with a broader tail down to 19 clues (mean 24.0, std 1.4). Thus cross-corpus experiments measure not only model architecture but also a *distribution shift* in how many digits are provided upfront—a useful stress test when interpreting blank-cell vs. full-puzzle accuracy.

Evaluation metrics. We primarily report *blank-cell accuracy* (among cells blank in the puzzle, the fraction whose predicted digit matches the solution) in Sec. 5; full-grid exact match is a stricter “solver” metric that we report where informative.

4. Methods

We study Sudoku with a shared supervised objective and a separate RL formulation. Let $k=m^2$ denote the board side length (classic Sudoku has $k=9$). Unless otherwise stated, all experiments use $k=9$; the same architectural blocks extend to other valid k by changing channel counts ($k+1$ input channels, k logits per digit class), tensor shapes $(\cdot, k, k)$, and (for GCNs) the peer graph over k^2 nodes.

Inference complexity. For fixed network depth and width, a single forward pass consists of a polynomial number of standard tensor operations in the number of cells $n=k^2$ (e.g., linear layers, sparse/structured message passing on $O(n)$ edges for Sudoku peers, or self-attention layers polynomial in n). Thus neural predictors provide per-instance inference whose asymptotic scaling is qualitatively different from worst-case exhaustive search on generalized instances [11].

All supervised models consume a one-hot clue tensor of shape $(k+1, k, k)$ and output per-cell logits organized as a tensor (k, k, k) : the first two axes index grid coordinates and the third indexes digit classes $\{1, \dots, k\}$.

4.1. Design rationale and alternatives

We compare four supervised inductive biases on the *same* input tensor: an MLP (no spatial structure), a CNN (local 3×3 neighborhoods), a Transformer (global self-attention over k^2 cells), and a GCN on the fixed Sudoku *peer graph* (row/column/block edges with residual message passing).

Table 1. MLP architecture (representative configuration). Shapes are shown for general order- k Sudoku; experiments use $k=9$.

Layer	Shape
One-hot input	$(k+1, k, k)$
Flatten	$((k+1)k^2)$
Linear + ReLU	(256)
Linear + ReLU	(256)
Linear	(k^3)
Reshape	(k, k, k)

This design directly exercises course themes—dense layers, convolutions, attention, and relational graphs—while holding the prediction task fixed (one-shot fill from clues).

We did *not* benchmark a classical DLX/CSP solver in our tables: exact methods already solve 9×9 instances and optimize a different objective (search cost and certificates) than learned inference.

For reinforcement learning we use PPO rather than pure behavioral cloning: Sudoku has a large factored action space (cell $\times$ digit) and sparse terminal rewards. Invalid placements are suppressed by masking logits to empty cells and Sudoku-valid candidates; optional MRV cell ordering further shrinks the search (Sec. 5.8). The actor-critic backbone is a Transformer so RL and the strongest supervised global model share the same token representation.

4.2. MLP baseline

The MLP flattens the clue tensor to a vector of dimension $(k+1)k^2$ and passes it through fully connected layers, then reshapes to (k, k, k) logits (Tab. 1). This baseline has minimal structural prior: any interaction between cells must be learned implicitly.

4.3. CNN encoder

The CNN keeps the grid layout and applies stacked 3×3 convolutions with batch normalization and ReLU, followed by 1×1 convolutions to map channels to k logits per spatial location (Tab. 2). This encodes a *local* translation-equivariant prior on the board (for very small k , kernel sizes may be adjusted).

4.4. Transformer encoder

We treat the board as a sequence of k^2 tokens: each cell receives a digit embedding plus a learnable positional embedding. A Transformer encoder stack [10] updates all tokens in parallel, and a linear readout predicts k logits per cell (Tab. 3). Self-attention enables direct long-range dependency modeling between any pair of cells.

Table 2. CNN architecture (representative configuration). Spatial tensors are $k \times k$; channel dimensions are for $k=9$.

Layer	Shape
Conv 3×3 + BN + ReLU	$(64, k, k)$
Conv 3×3 + BN + ReLU	$(128, k, k)$
Conv 3×3 + BN + ReLU	$(256, k, k)$
Conv 1×1 + ReLU	$(128, k, k)$
Conv 1×1 + ReLU	$(64, k, k)$
Conv 1×1 (logits)	(k, k, k)

Table 3. Transformer architecture (representative configuration).

Component	Shape
Token + position embedding	(k^2, d)
Transformer encoder (L layers, H heads)	(k^2, d)
Linear readout per cell	(k^2, k)
Reshape	(k, k, k)

4.5. Graph convolutional network (GCN)

We construct a fixed graph with k^2 nodes (one per cell). Undirected edges connect cells that share a row, a column, or the same $\sqrt{k} \times \sqrt{k}$ block (the Sudoku “peer” relation), plus self-loops. Let $\tilde{\mathbf{A}}$ denote the symmetrically normalized adjacency $\mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ (including self-loops in $\mathbf{A}$). Node features begin from the $(k+1)$ -dimensional one-hot clue vector at each cell. Stacking many peer-aggregation steps without stabilization can oversmooth node states and drive deep ReLU stacks toward near-constant logits (cross-entropy near $\ln k$ with little learning signal). We therefore use a lightweight residual block with layer normalization after aggregation:

$$\mathbf{H}^{(\ell+1)} = \mathbf{H}^{(\ell)} + \sigma\left(\mathbf{W}^{(\ell)} \text{LN}\left(\tilde{\mathbf{A}}\mathbf{H}^{(\ell)}\right)\right), \quad (1)$$

with σ a pointwise nonlinearity, followed by a linear map to k logits per node and a reshape to (k, k, k) . Compared to a CNN, the GCN’s receptive field is defined by *constraint structure* rather than square spatial neighborhoods; compared to a Transformer, it avoids all-to-all attention and scales linearly in the number of edges for this fixed graph.

4.6. Reinforcement learning with PPO

We also formulate Sudoku as a Markov decision process: at each step the agent selects an empty cell and a digit to write. The environment validates row/column/box constraints and returns shaped rewards (small positive reward for legal placements, negative reward for illegal moves, and a completion bonus).

Table 4. GCN configuration (representative).

Component	Value
Nodes	k^2 (one per cell)
Edges	Sudoku peer graph + self-loops; fixed $\tilde{\mathbf{A}}$
Input / node	$k+1$ (one-hot clue)
Hidden dim.	128
MP layers	6 (residual + LN)
Readout	k logits/node $\rightarrow (k, k, k)$

Policy and value function. We use an actor-critic with a Transformer backbone over the k^2 cell tokens (digit + positional embeddings). The actor factorizes actions into (i) a categorical distribution over empty positions and (ii) a categorical distribution over digits at the chosen cell. We mask invalid logits (non-empty positions; digits inconsistent with Sudoku candidates at the chosen cell) and renormalize.

Optimization. We optimize with PPO [9]: clipped surrogate objective, value loss, and entropy bonus, with advantage estimates from discounted returns. This paradigm is complementary to supervised one-shot prediction: the policy is trained under its own interaction distribution rather than only imitating a teacher solution in a single forward pass.

4.7. Training protocol (supervised)

Unless otherwise stated, supervised models are trained with AdamW [4] on cross-entropy to the solution digits, using the same preprocessing pipeline described in Sec. 3. We track blank-cell accuracy on a held-out split and use early stopping based on validation improvement; gradient norms are clipped (max norm 1.0) for stability.

5. Experiments

This section reports our completed runs under a fixed protocol: $N=20,000$ training puzzles per dataset, an 80%/20% train/validation split on that subsample, and two validation metrics reported in percent: blank-cell accuracy (share of initially empty cells correct) and puzzle-wise accuracy (share of puzzles with a fully correct 81-cell grid).

5.1. Experimental protocol

We train supervised models with the same loss and comparable optimization defaults (AdamW, learning rate 10^{-3} , weight decay 10^{-2} , batch size 64). Unless noted, supervised models are trained for 50 epochs on the subsample (with early stopping keyed to validation blank-cell accuracy where used). Metrics are computed on the held-out validation fraction of the *same* subsample used for training. For reinforcement learning, we evaluate PPO with the same two

Table 5. Default supervised optimization settings (shared across MLP/CNN/Transformer/GCN unless noted).

Setting	Value
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	10^{-2}
Batch size	64
Epochs	50 (early stopping on val. blank acc. where used)
Loss (default)	Cross-entropy on all 81 cells
Train/val split	80%/20% on the 20,000-row subsample

Table 6. Supervised validation results (% , $N=20,000$ subsample per dataset, 50 epochs, cross-entropy on all 81 cells). Each corpus column reports blank-cell / puzzle-wise accuracy.

Model	sudoku-1m	3m-easy	3m-hard
MLP	55.8% / 0.0%	32.5% / 0.0%	30.7% / 0.0%
CNN	30.6% / 0.0%	22.1% / 0.0%	21.5% / 0.0%
Transformer	95.6% / 30.1%	52.8% / 0.0%	52.9% / 0.0%
GCN	97.2% / 40.6%	58.1% / 0.0%	53.6% / 0.0%

metrics on the validation split after training on the corresponding 20,000-puzzle list, using a supervised-style rollout decoder (final filled grid compared to labels).

5.2. Supervised results (three corpora)

Metrics. *Blank-cell accuracy* is the percentage of initially empty cells whose predicted digit matches the solution; *puzzle-wise accuracy* is the percentage of puzzles whose full 81-cell prediction matches the label (a stricter “solver” metric). Tab. 6 reports both metrics for four supervised architectures trained with cross-entropy on *all* 81 cells (including given clues), on *sudoku-1m* and on the difficulty-stratified *sudoku-3m-easy* / *sudoku-3m-hard* exports defined in Sec. 3.

Observations on sudoku-1m. The GCN attains the highest blank-cell accuracy (97.2%) and the best puzzle-wise rate (40.6%), closely followed by the Transformer on blanks (95.6%) with 30.1% puzzle-wise solves. The MLP reaches 55.8% blank accuracy and the CNN 30.6%; neither achieves a non-zero puzzle-wise rate on this split. Small receptive fields without explicit long-range structure thus struggle even on the relatively concentrated *sudoku-1m* clue distribution (Sec. 3).

Observations on sudoku-3m-easy and sudoku-3m-hard. All models degrade relative to *sudoku-1m*, reflecting both distribution shift and fewer upfront clues in the 3m subsets. The GCN remains

Table 7. Sudoku **complete-board** accuracy (%) **reported by** Zhang et al. [12] (Table 2). DDReasoner-SL is supervised diffusion; DDReasoner-RL adds constraint-guided policy optimization. The Baseline row is the SolverNN submodule used for comparison in that work.

Model	big kaggle	min. 17	mult. sol	satnet
Baseline	62.68%	0.00%	46.70%	24.00%
DDReasoner-SL	78.19%	8.03%	96.30%	70.60%
DDReasoner-RL	97.79%	18.25%	100.00%	92.60%

strongest on both exports (58.1% easy, 53.6% hard), ahead of the Transformer (52.8% / 52.9%). MLP and CNN drop to the 21%–33% range on the 3m splits. Notably, blank-cell accuracy on easy and hard is similar for the Transformer ($\approx 53\%$) despite very different clue-count profiles (Sec. 3), while the GCN retains a modest edge on the easy subset.

5.3. External reference: diffusion + RL (DDReasoner)

Our project focuses on “direct” supervised predictors (MLP/CNN/Transformer/GCN) and a PPO policy on the Sudoku environment. To situate these choices relative to recent neuro-symbolic learning, Tab. 7 transcribes Sudoku results **as reported by Zhang et al. [12]** (their Table 2). Their metric is **complete-board accuracy** (% of puzzles with all cells correct and rule-consistent), which is stricter than blank-cell accuracy and is evaluated on the public splits/datasets described in their paper (e.g., *big kaggle*, *minimal 17*, *multiple sol*, *satnet data*).

5.4. Attention and influence maps

To inspect how global (Transformer) and graph-structured (GCN) models allocate credit to other cells when predicting the center blank (5, 5), we visualize query-to-all-cell maps at inference time on a representative *sudoku-1m* validation puzzle (29 clues; Figs. 3 and 4). The query token is index $4 \times 9 + 4$ (blue ring in each heatmap). For the Transformer we read out self-attention weights (max over heads, percentile contrast stretch) from the last layer and from the mean over layers $L_{1:4}$. The GCN has no attention matrix; we show the normalized peer-graph row at the query node, gradient attribution to the input one-hot at (5, 5), and a layerwise peer-message influence heuristic.

Interpretation. Both architectures allocate most of their query-to-cell mass to *Sudoku-relevant* neighborhoods rather than to arbitrary distant cells, which is evidence that they internalize the puzzle constraints during training. For the Transformer, the layer-4 map is sharply peaked inside the query’s 3×3 block, while averaging attention over $L_{1:4}$ yields a clear row–column–block pattern centered on

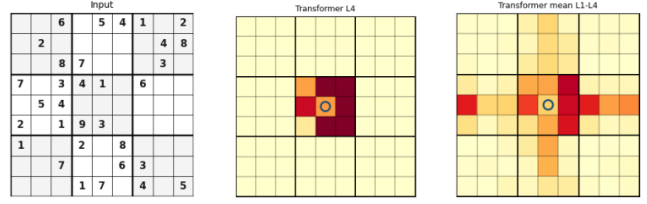


Figure 3. Transformer query-to-all-cell maps for (5, 5) on a *sudoku-1m* validation puzzle (29 clues). **Left:** input (bold = given). **Center:** layer-4 self-attention. **Right:** mean attention over layers 1–4.

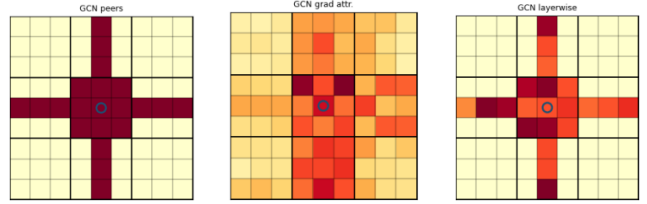


Figure 4. GCN influence maps for the same query cell (5, 5). **Left:** structural peers from $\tilde{A}$. **Center:** gradient attribution. **Right:** accumulated layerwise peer influence.

(5, 5)—the same peer structure a human solver would consult before placing a digit. For the GCN, the peer-graph row (left) is fixed by construction, but the *learned* maps are informative: layerwise influence (right) recovers the full row/column/block cross, indicating that message passing over $\tilde{A}$ is used to propagate constraint information; gradient attribution (center) is more localized yet still emphasizes the query cell and its immediate peers. Taken together, these maps support the quantitative results in Tab. 6: relational inductive bias (GCN) and global attention (Transformer) are both capable of learning Sudoku rules, with the GCN’s graph prior making the constraint structure explicit and the Transformer’s depth-averaged attention revealing the same structure after several encoder layers. They do not guarantee full-puzzle correctness—as Sec. 5.5 shows, high blank-cell accuracy can coexist with occasional long-range violations—but they confirm that neither model is relying on unstructured, board-wide averaging.

5.5. Qualitative failure modes

Even the strongest supervised model (GCN) occasionally fails full-puzzle exact match on *sudoku-1m*. Fig. 5 shows one validation example: five initially blank cells are mispredicted (red boxes in the GCN panel) despite locally plausible digits. The puzzle has 29 given clues; errors include confusing digits 4/9 and 5/7 in different regions, consistent with occasional long-range constraint violations rather than uniform random guessing.

Puzzle (input)								GCN prediction								Ground truth													
		6		5	4	1	2		3	7	6	8	5	4	1	9	2		3	7	6	8	5	4	1	9	2		
	2									5	2	9	3	6	1	7	4	8		5	2	9	3	6	1	7	4	8	
		8	7					3			4	1	8	7	2	9	5	3	6		4	1	8	7	2	9	5	3	6
7		3	4	1		6				7	8	3	4	1	2	6	9	9		7	8	3	4	1	2	6	5	9	9
		5	4							9	5	4	6	8	7	2	1	3		9	5	4	6	8	7	2	1	3	
2		1	9	3						2	6	1	9	3	7	8	5	4		2	6	1	9	3	5	8	7	4	
1			2		8					1	3	5	2	4	8	9	6	7		1	3	5	2	4	8	9	6	7	
		7			6	3				8	4	7	5	4	6	3	2	1		8	4	7	5	4	6	3	2	1	
			1	7		4	5			6	9	2	1	7	3	4	8	5		6	9	2	1	7	3	4	8	5	

Figure 5. GCN failure on `sudoku-1m` validation (29 clues). Left: input puzzle (bold = given). Center: model fill; red highlight = wrong blank (5 cells). Right: ground-truth solution.

Table 8. Supervised validation with loss on blank cells only (%), same splits as Tab. 6). Each corpus column reports blank-cell / puzzle-wise accuracy.

Model	sudoku-1m	3m-easy	3m-hard
MLP	63.5% / 0.0%	40.8% / 0.0%	38.5% / 0.0%
CNN	31.2% / 0.0%	22.7% / 0.0%	22.0% / 0.0%
Transformer	96.1% / 0.0%	49.8% / 0.0%	38.2% / 0.0%
GCN	97.2% / 4.1%	56.3% / 0.0%	52.5% / 0.0%

5.6. Reinforcement learning (quantitative summary)

We trained PPO with constraint-aware action masking on 20,000-puzzle lists. On `sudoku-1m` with the default guided mode (MRV + candidate mask), validation blank-cell accuracy is 99.97% and puzzle-wise accuracy 99.95% (Tab. 9); training on the full `sudoku-3m` parent subsample (not the easy/hard exports in Tab. 6) yields 26.7% blank-cell accuracy under the same decoder. The 2×2 ablation in Sec. 5.8 shows that either heuristic alone is insufficient: without both MRV and masking, accuracy stays near chance ($\approx 14\%$ – 19%). Readers should therefore interpret near-perfect in-distribution RL numbers as “policy learning under heavy structural guidance” rather than unconstrained end-to-end grid filling.

5.7. Ablation: loss on blank cells only

A natural supervised variant is to compute cross-entropy only on initially empty cells, matching the evaluation metric more directly. Tab. 8 repeats the same 50-epoch / 20,000-puzzle protocol with *blank-only* loss.

Comparison to all-cell loss (Tab. 6). On `sudoku-1m`, blank-only loss *helps* the MLP on blanks (63.5% vs. 55.8%) and leaves the Transformer essentially unchanged on blanks (96.1% vs. 95.6%); the GCN blank accuracy is also unchanged (97.2% vs. 97.2%). Puzzle-wise accuracy nonetheless collapses under blank-only loss for the Transformer (30.1% $\rightarrow$ 0.0%) and falls for the GCN (40.6% $\rightarrow$ 4.1%); MLP and CNN remain at zero on both

Table 9. PPO 2×2 ablation on `sudoku-1m` (%), $N=20,000$ train subsample, held-out validation).

Mode	Blank-cell (%)	Puzzle-wise (%)
guided (MRV + mask)	99.97%	99.95%
mrsv_only	18.6%	0.0%
mask_only	16.1%	0.0%
free	13.9%	0.0%

loss variants—so aligning the training objective with blank-cell accuracy does not by itself yield full-puzzle solves. On the 3m exports, the Transformer recovers blank accuracy relative to its all-cell run (49.8%/38.2% vs. 52.8%/52.9%), while puzzle-wise accuracy stays at zero for every model on both loss settings.

5.8. Ablation: PPO action guidance (2×2)

We ablate how much of PPO’s strong `sudoku-1m` performance comes from *where* to act (minimum remaining values, MRV, for cell choice) versus *what* digit to write (masking logits to Sudoku-valid candidates at the chosen cell). Four modes form a 2×2 design on the same $N=20,000$ `sudoku-1m` subsample and validation split as supervised runs: `guided` (MRV + candidate mask), `mrsv_only`, `mask_only`, and `free` (neither heuristic). Evaluation uses the same rollout decoder as in Sec. 4.6 (final grid vs. label).

Interpretation. Only the fully guided policy (`guided`) approaches ceiling on both metrics; removing either MRV or candidate masking collapses blank-cell accuracy to the 14%–19% range with zero puzzle-wise solves. The near-random band for the three weakened modes indicates that, under our episode budget, the policy does not learn reliable sequential filling without both structural priors—consistent with the masking-heavy RL narrative in Sec. 4.6. `mrsv_only` slightly exceeds `mask_only` and `free`, suggesting that fixing *which* empty cell to fill next is somewhat more informative than digit masking alone when position choice is left to the network.

5.9. Discussion: threats to validity and reproducibility

Subsample bias. Performance depends on how many puzzles are trained on and how hard they are; different Sudoku corpora (e.g., `sudoku-1m` vs. `sudoku-3m`) vary in clue-count and difficulty filtering. We mitigate this by fixing $N=20,000$ subsamples and a single split seed across models.

Metric choice. Blank-cell accuracy can be inflated by “easy blanks” correlated with clues; we therefore report puzzle-wise (full-grid) accuracy alongside blanks in Tabs. 6, 8 and 9.

Compute fairness. All architectures were trained for the same epoch budget; per-step FLOPs and wall-clock differ across MLP, CNN, Transformer, and GCN.

Reproducibility. We fix $N=20,000$ subsamples, an 80%/20% split, and the optimization settings in Tab. 5 across all reported runs. Our PyTorch training notebook and experiment scripts are available at <https://github.com/FrankChen0205/Sudoku>.

6. Conclusion

We motivated neural Sudoku solvers from a scalability perspective: classical exact methods can be extremely accurate but can face sharply growing worst-case search cost on larger generalized instances, whereas fixed-depth neural networks offer polynomial-time *inference* that amortizes learning across puzzles. Within that framing, Sudoku becomes a clean testbed for asking *how different neural architectures reason*: how they propagate row/column/block constraints through inductive bias (local convolutions, global attention, graph message passing) and through alternative training objectives (imitation vs. PPO rollouts).

Our quantitative evidence is obtained on `sudoku-1m` and on difficulty-stratified `sudoku-3m-easy` / `sudoku-3m-hard` exports, with identical preprocessing aside from blank-token normalization. Under a shared 50-epoch supervised recipe on 20,000 puzzles per corpus, the GCN achieves the best blank-cell accuracy on all three splits (97.2% on `sudoku-1m`, 58.1% easy, 53.6% hard), with the Transformer close behind on `sudoku-1m` (95.6%) and on both 3m subsets ($\approx 53\%$). The CNN underperforms the MLP everywhere in this budget—consistent with the need for explicit long-range structure beyond small receptive fields. An ablation with loss on blank cells only (Sec. 5.7) improves the MLP on `sudoku-1m` blanks (55.8% $\rightarrow$ 63.5%) while the Transformer retains strong blank accuracy (96.1%); however, puzzle-wise solves still vanish for the Transformer (30.1% $\rightarrow$ 0.0%), and the GCN puzzle-wise rate drops from 40.6% to 4.1%. PPO attains near-perfect validation on `sudoku-1m` only when both MRV cell choice and candidate masking are enabled (Sec. 5.8); removing either prior drops accuracy to $\approx 14\%$ – 19% . On the full `sudoku-3m` subsample, guided PPO still degrades (26.7% blank accuracy), paralleling supervised failure under distribution shift.

What we learned. Blank-cell accuracy alone can mask failure to complete puzzles: GCN and Transformer excel on `sudoku-1m` blanks but still miss full-grid exact match on many validation boards (Fig. 5). Aligning the training loss with the metric (blank-only cross-entropy) can raise blank-cell scores without producing full puzzles: the Transformer keeps 96.1% blank accuracy on `sudoku-1m` but loses all puzzle-wise solves under blank-only loss. For PPO, hard

constraint priors (MRV + masking) are not optional extras but dominate learning under our budget.

Future work. Promising extensions include multi-seed training curves, constraint-violation rates on model outputs, efficiency tables (parameters and ms/step), and an independent rerun of diffusion+RL baselines such as DDReasoner [12] under matched metrics.

References

- [1] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinícius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018. 2
- [2] Donald E. Knuth. Dancing links, 2000. 1, 2
- [3] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 2
- [4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. 5
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 2
- [6] Rasmus Palm, Ulrich Paquet, and Ole Winther. Recurrent relational networks. In *Advances in Neural Information Processing Systems*, 2018. 2
- [7] Bryan Park and contributors. Sudoku puzzles and solutions dataset. Kaggle dataset, 2018. <https://www.kaggle.com/datasets/bryanpark/sudoku>. 1, 2
- [8] Colin Radcliffe. 3 million sudoku puzzles with ratings. Kaggle dataset, 2020. <https://www.kaggle.com/datasets/radcliffe/3-million-sudoku-puzzles-with-ratings>; columns puzzle, solution, clues, difficulty. 2
- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 2, 5
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. 2, 4
- [11] Takayuki Yato and Takahiro Seta. Complexity and completeness of finding another solution and its application to puzzles, 2003. IPSJ SIGALGS Technical Report AL-2003-0004. 1, 2, 3
- [12] Xuan Zhang, Zhijian Zhou, Weidi Xu, Yanting Miao, Chao Qu, and Yuan Qi. Constraints-guided diffusion reasoner for neuro-symbolic learning, 2025. 2, 6, 8